MICROPROCESSORS PROJECT LAB · FINAL REVIEW

Smart Classroom Automation System

Occupancy-based lighting, RFID entry logging and fan control on a bare-metal STM32F401RE

Tejaswi
Motor driver and fan

Ayush
RFID integration

Tejas
DHT11 and RTC

Tanmay
System, sensing, integration

School of Electrical and Electronics Engineering · MIT Manipal · 29/10/2026

AGENDA

Presentation flow

01	**System overview, occupancy sensing and lighting** Problem, architecture, firmware structure, ultrasonic zones, debounce, lighting	Tanmay
02	**RFID entry at the door** RC522 on SPI2, ISO 14443-A card read, roster and lockout, link to the system	Ayush
03	**Motor driver and fan control** IRLZ44N low-side stage, TIM1 PWM at 20 kHz, button fan levels, RFID gating	Tejaswi
04	**Environment sensing and real-time clock** DHT11 single-wire protocol, RTC clock source, backup domain, serial time setting	Tejas
05	**Interrupts, user interface and results** Interrupts, one-button screens, roster, class sessions, OLED, testing, demo	Tanmay

INTRODUCTION

Problem and objectives

Classroom lights and fans are often left running after the room empties.

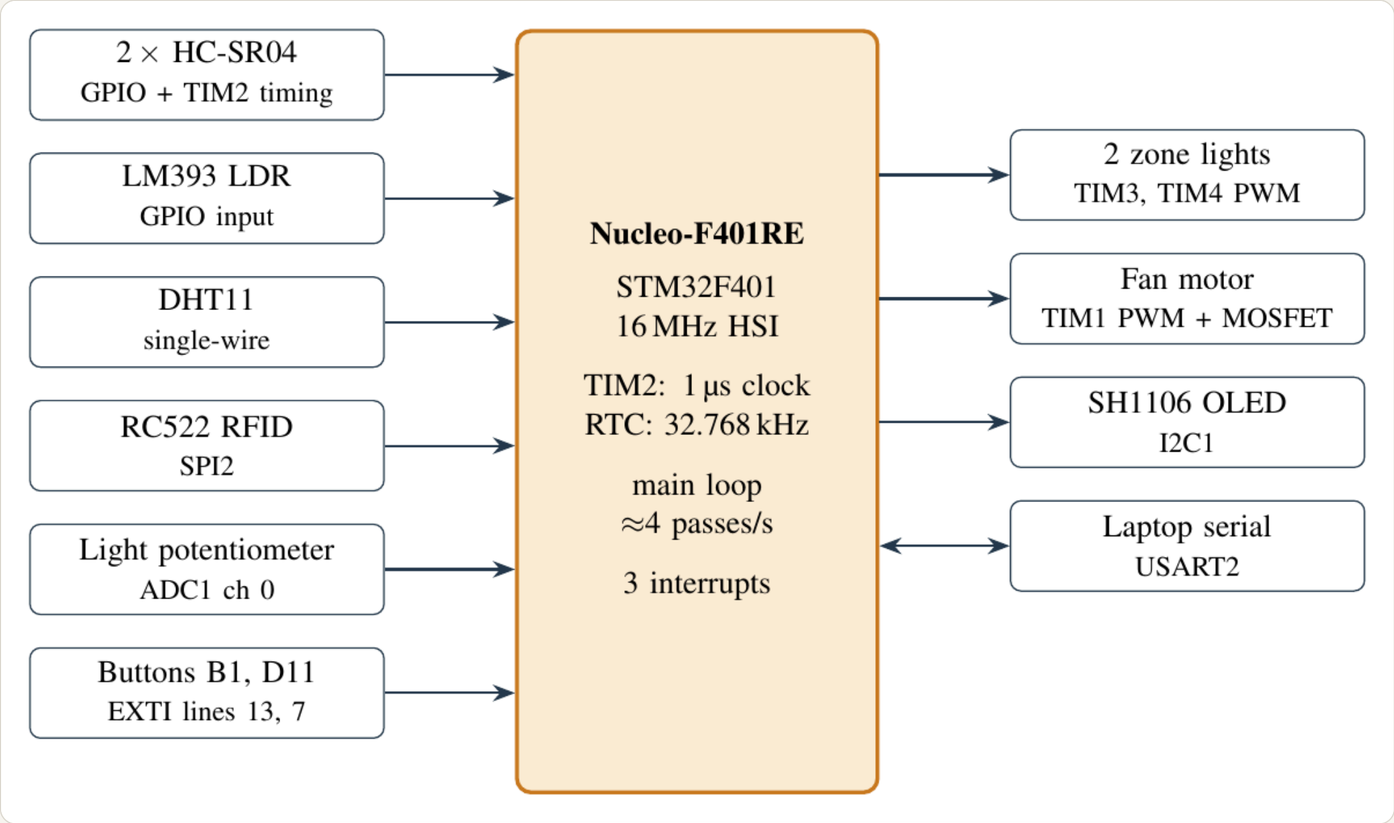
Manual switching depends on the last person to leave, and a fixed timer cannot tell whether anyone is present. The loads should follow actual occupancy, ambient light and the number of people inside.

Objectives

1. Detect occupancy in two zones using ultrasonic ranging
2. Switch and dim zone lights by occupancy and ambient light
3. Record entry and exit at the door with RFID cards
4. Run the fan only while someone is inside, at a set speed
5. Display temperature, humidity, date and time
6. Write all firmware at register level, using interrupts

ARCHITECTURE

System block diagram



Inputs

Two ultrasonic sensors, LDR module, DHT11, RC522 reader, light potentiometer, two push buttons

Outputs

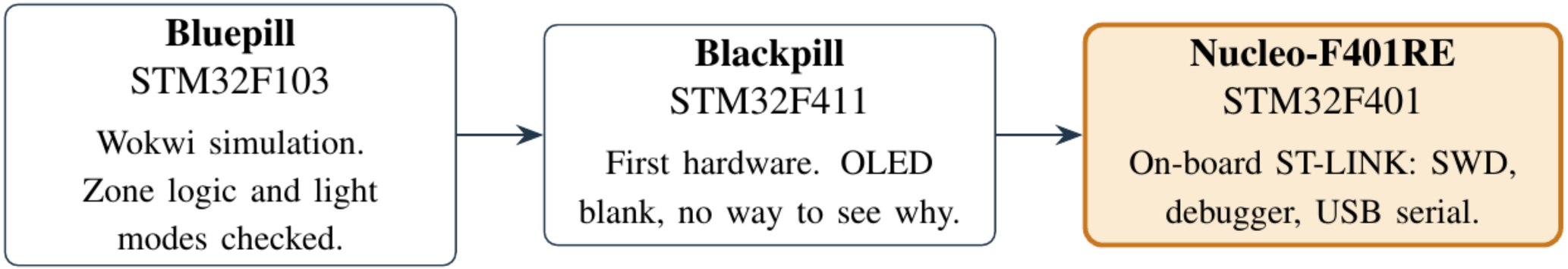
Two PWM zone lights, PWM fan through a MOSFET, OLED display, serial log

Core

16 MHz internal oscillator, no PLL. TIM2 counts microseconds for all timing.

DEVELOPMENT PATH

How the platform was chosen



Simulation first

Zone logic and light modes were checked on a simulated Bluepill in Wokwi before any hardware was wired.

Arduino build as a wiring check

An Arduino sketch confirmed the sensor and display wiring. The register-level port replaced it.

Three zones reduced to two

A persistent wiring fault on the third ultrasonic sensor led us to finalise the design with two zones.

FIRMWARE

Bare-metal structure and peripheral map

- Each peripheral is configured directly through its registers using the CMSIS device header. No HAL and no Arduino core.
- Clock, timer and pin settings are explicit in the source.
- Decision logic is kept apart from hardware access, so it can be compiled and tested on a PC.

`main.c`
Drivers and main loop

`logic.h`
Pure decision logic

`extras.h`
Sessions, log, graphs

Peripheral	Function	Setting
TIM2	Microsecond time base	1 MHz, free-running
TIM3, TIM4	Zone A and B lights	1 kHz PWM
TIM1	Fan MOSFET gate	20 kHz PWM
ADC1	Light knob	Channel 0, 8-sample mean
SPI2	RC522 RFID reader	1 MHz, mode 0
I2C1	SH1106 OLED	100 kHz
USART2	Log and commands	115200 baud, RX IRQ
RTC	Date and time	32.768 kHz crystal
EXTI13	Blue button B1	Falling-edge IRQ
EXTI7	Menu button D11	Both-edge IRQ

FIRMWARE

One pass of the main loop, about 0.25 s

01
Ping zones
Zone A, then zone B, 60 ms apart

02
DHT11
Read every 2 s; re-probe OLED and reader

03
Serial input
Parse typed time and date commands

04
Buttons
Fan level; menu tap or hold

05
Knob
Light pot, 8-sample mean

06
Occupancy
Debounce each zone’s reading

07
Lights
Set the PWM duty of each zone

08
Fan
Duty from headcount and fan level

09
Stale check
Clear the roster after 120 s of empty zones

10
Report
Draw screen, send changed pages, log

The RFID reader is polled during every wait and between display updates, so a card tap is not missed while the loop runs.

OCCUPANCY SENSING

Ultrasonic distance measurement

Measurement sequence

1. Drive TRIG high for 10 µs.
2. Wait up to 5 ms for ECHO to rise, else report NO ECHO.
3. Time the ECHO pulse on TIM2; beyond 25 ms, report FAR.
4. Distance in cm = echo time in µs ÷ 58.
5. A distance under 100 cm counts as a person present.

58 µs/cm

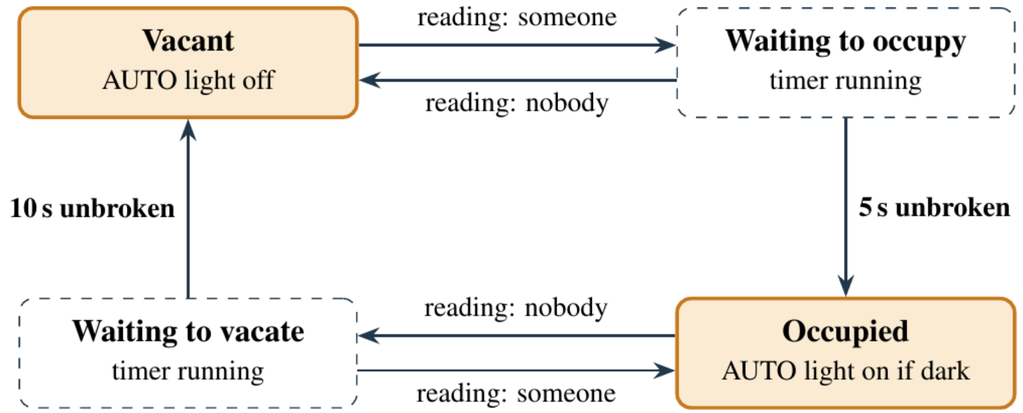
Sound travels about 0.0343 cm per µs and covers the distance twice: 2 ÷ 0.0343 ≈ 58 µs per cm.

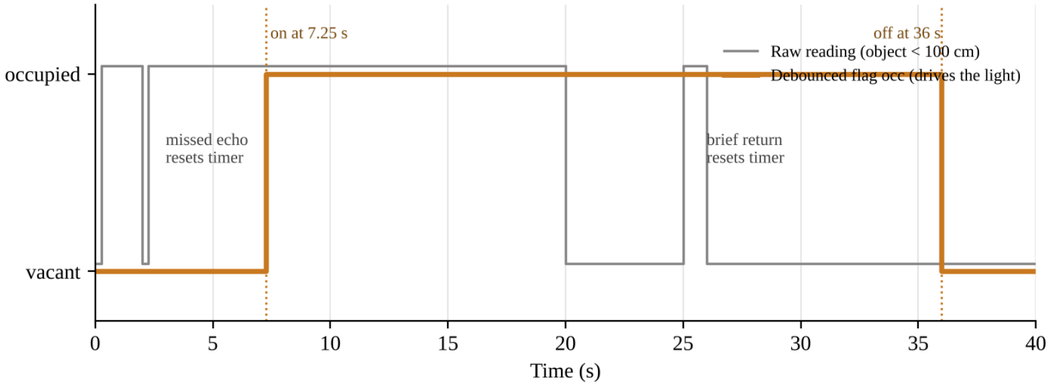
Zone	TRIG	ECHO
A	PA10 (D2)	PB3 (D3)
B	PB5 (D4)	PB4 (D5)

Sensors run from 5 V. The two zones are pinged 60 ms apart so one sensor’s echo is not read by the other.

OCCUPANCY SENSING

Asymmetric debounce of each zone





5 s to occupy

A person walking past does not switch the light on.

10 s to vacate

A seated person who briefly drops out of range keeps the light.

Timer restarts

Any contrary reading clears pending_since, so only unbroken runs count.

LIGHTING

Zone lights: modes, PWM and ambient light

Setting	Light output
OFF	Always off
AUTO (default)	Knob brightness when occupied and dark
DIM	Fixed low level, 10 % duty
MAX	Knob brightness at all times

Each zone’s mode is set at compile time; the final build runs both zones in AUTO. There is no run-time override: the blue button now sets the fan level.

PWM outputs

TIM3 CH2 on PC7 (D9), TIM4 CH1 on PB6 (D10). 16 MHz ÷ 16 ÷ 1000 = 1 kHz, duty 0 to 999.

Ambient light

LM393 module output on PB0, internal pull-up. HIGH means dark.

Brightness knob

PA0 (A0), ADC1 channel 0. On the HOME screen the 12-bit reading maps to duty 50 to 999, with a small dead band against flicker.

02

RFID entry at the door

Reading ISO 14443-A cards with the RC522 over SPI, keeping a roster of who is inside, and passing the headcount to the rest of the system.

RFID · HARDWARE

RC522 reader on SPI2

RC522 pin	Nucleo	Role
SCK	PB13	SPI2 clock
MISO	PB14	Data to the MCU
MOSI	PB15	Data to the reader
SDA (SS)	PB12	Chip select, by software
RST	PC5	Hardware reset
VCC	3V3	3.3 V only, never 5 V

SPI settings

Master, 16 MHz ÷ 16 = 1 MHz, mode 0 (CPOL 0, CPHA 0), 8-bit, MSB first, software slave select.

Register address byte



Bit 7 is 1 to read and 0 to write; bits 6 to 1 hold the register address; bit 0 is always 0.

At start-up: pulse RST low, issue a soft reset (0x0F), then read VersionReg (0x37). A value of 0x00 or 0xFF means no reader is connected.

RFID · PROTOCOL

Reading a card, ISO 14443-A

01

Configure

Timer ticks of 25 µs, reload 320: no answer in 8 ms means no card. 100 % ASK, antenna on.

02

REQA

Send 0x26 as a 7-bit short frame. An idle card answers with its 2-byte ATQA.

03

Anticollision

Send 0x93 0x20. The card returns 4 UID bytes and a check byte, BCC.

04

Validate

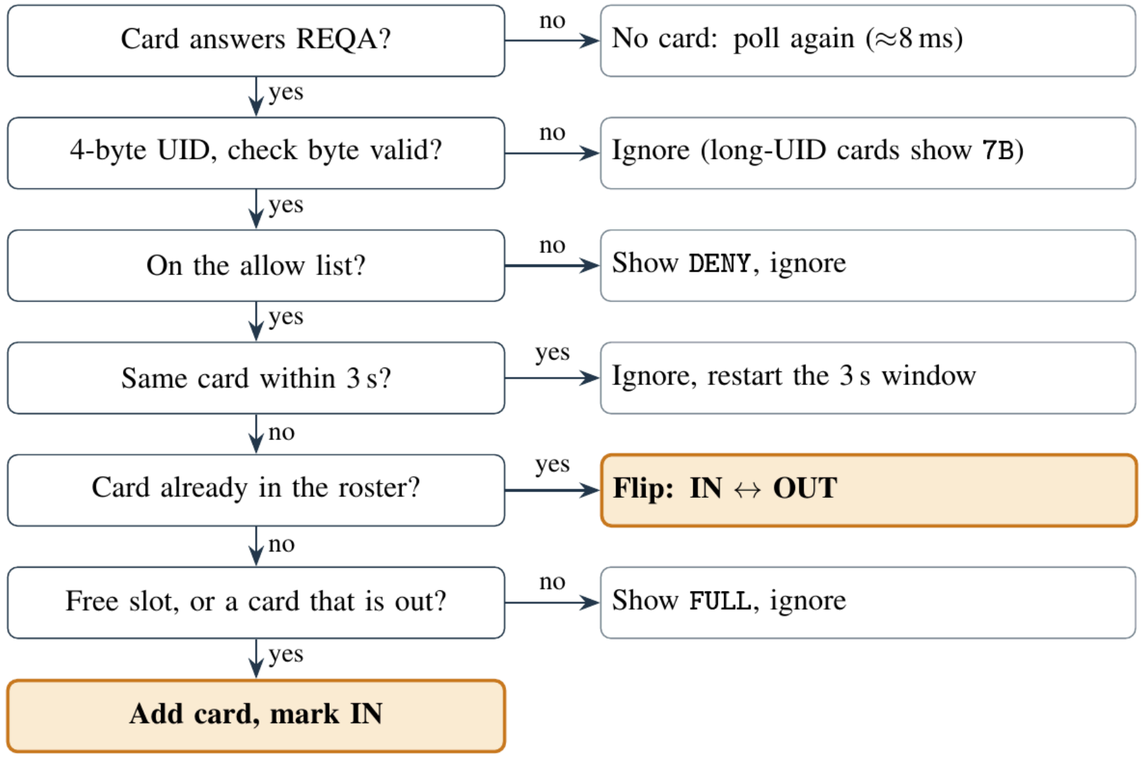
BCC must equal the XOR of the UID bytes. A first byte of 0x88 marks a 7-byte UID, not supported.

Every exchange with the card

```
Idle → clear IRQ flags → flush FIFO → load bytes → Transceive (0x0C) → StartSend → wait for RxIRq or IdleIRq (TimerIRq = no card) → check ErrorReg → read FIFO
```

RFID · ROSTER

Turning taps into a headcount



Toggle per card

Up to 8 cards. Each tap flips a card between IN and OUT.

3 s lockout

A card held on the reader keeps restarting the window, so one tap is one event.

Allow list

UIDs in ALLOWED_UIDS; a list of only 0 accepts any card. Others show DENY.

Logged

Each event appears on the OLED and on serial with the RTC time.

RFID · INTEGRATION

How the headcount drives the room

OCC only

Lights stay sensor-driven

A tap never switches a light. Lights follow the zone sensors, so a tap at the door cannot light an empty zone.

IN > 0

Fan gating

The fan runs only while at least one card is IN, at the level set by the blue button. Member A covers the driver next.

120 s

Stale reset

If cards are IN but both zones stay vacant for 120 s, the roster is cleared. The timer starts 30 s after the last tap-in. This covers people who leave without tapping out.

Limitations

Only 4-byte UIDs are read. The count depends on people tapping on the way out; the stale reset only limits the error. The roster holds 8 cards.

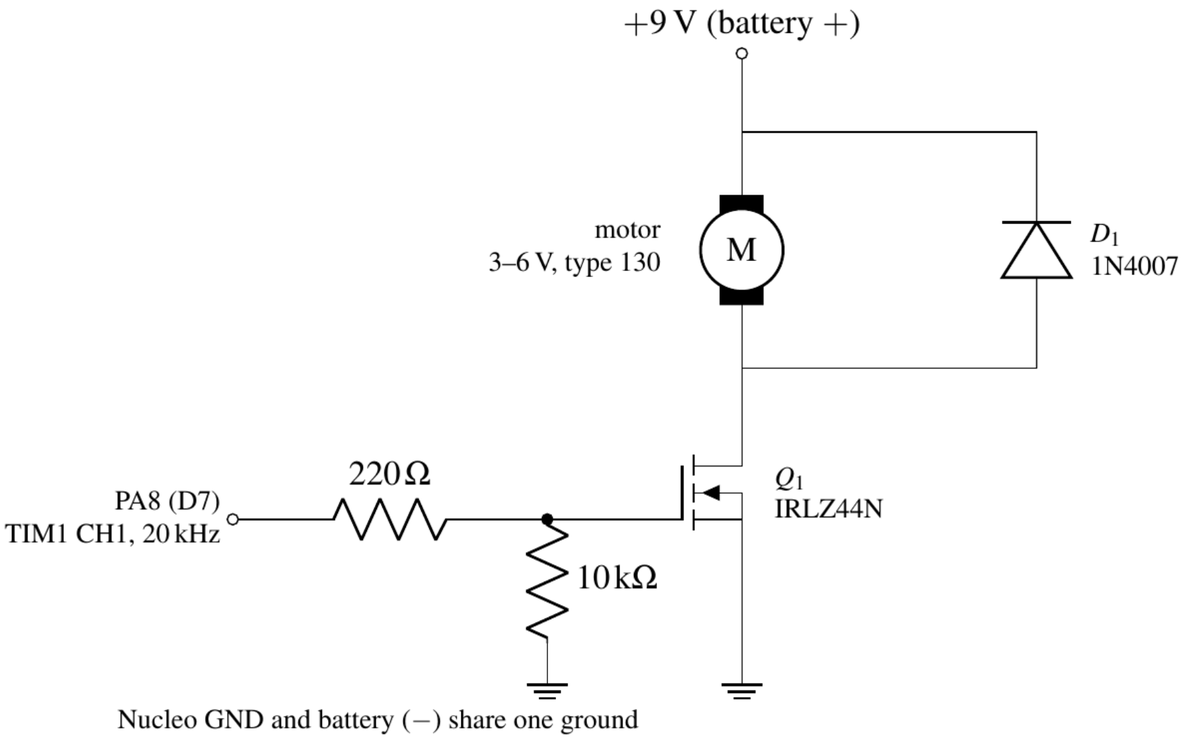
03

Motor driver and fan control

Switching a 9 V motor from a 3.3 V pin with a logic-level MOSFET, generating the PWM on TIM1, and deciding when and how fast the fan runs.

MOTOR DRIVER · CIRCUIT

Low-side MOSFET driver stage



Part	Value	Purpose
Q1	IRLZ44N	Switch
D1	1N4007	Flyback path
Gate R	220 Ω	Limit current
Pull-down	10 kΩ	Gate off at reset
Supply	9 V battery	Motor power
Motor	Type 130, 3–6 V	Fan

Battery negative is joined to the Nucleo ground. Without a shared ground the gate voltage has no reference.

MOTOR DRIVER · DESIGN

Why each component is there

Low-side switching

The source sits at ground, so the 3.3 V pin sets the gate–source voltage directly. A high-side N-channel switch would need a gate voltage above 9 V.

Logic-level MOSFET

The IRLZ44N has a gate threshold of 1 to 2 V and an on-resistance specified at 4 to 5 V drive. At 3.3 V it is not fully enhanced, but it conducts well enough for the small motor.

Flyback diode

When the MOSFET turns off, the motor’s inductance keeps the current flowing. The diode gives it a path back to +9 V and clamps the spike at the drain.

Gate resistors

220 Ω limits the pin current while the gate capacitance charges and damps ringing. 10 kΩ holds the gate low during reset, so the fan cannot start by itself.

Known improvement: the 1N4007 is a slow-recovery rectifier. A Schottky diode would suit 20 kHz switching better.

MOTOR DRIVER · PWM

Generating the fan PWM on TIM1

TIM1 channel 1 on PA8 (D7)

```
GPIOA PA8 → AF1 (TIM1_CH1)
PSC = 0, ARR = 799
→ 16 MHz ÷ 800 = 20 kHz
CCMR1 OC1M = 110 (PWM mode 1)
OC1PE = 1 (preload)
CCER CC1E = 1
BDTR MOE = 1 (main output enable)
CR1 ARPE | CEN
```

TIM1 is an advanced-control timer: its outputs stay off until MOE is set.

20 kHz

Above the audible range, so the motor does not whine.

60 % cap

9 V × 0.6 ≈ 5.4 V average, within the motor’s 3–6 V rating.

`CCR1 = pct × 60 × 8 ÷ 100`

100 % fan speed gives CCR1 = 480 of 800.

MOTOR DRIVER · CONTROL

When the fan runs, and how fast

0·33·66·100

Fan level button

Each press of the blue button B1 steps the level and wraps back to 0. Starts at 0 after a reset. Driven as 35, 68 and 100 % of the allowed speed, because the motor stalls below 35 %.

Decision table

Level	Card IN	Fan
0	Either	Off
33, 66, 100	No	Off
33, 66, 100	Yes	Runs at that level

The OLED shows FAN:66 RUN, or FAN:66 STOP while nobody is tapped in.

0.3 s

Kick-start

A stopped motor may not start at 35 %. When the command rises from zero, the fan runs at full allowed duty for 0.3 s, then settles to the chosen level.

04

Environment sensing and timekeeping

Reading temperature and humidity from the DHT11 over a single wire, and keeping date and time with the on-chip real-time clock.

DHT11 · PROTOCOL

Single-wire temperature and humidity read



Start

The MCU pulls the line low for 20 ms, then releases it.

Response

The sensor pulls low about 80 µs, then high about 80 µs.

40 data bits

50 µs low, then high: 26–28 µs is 0, about 70 µs is 1. Threshold 45 µs.

Checksum

Byte 4 = low 8 bits of bytes 0 to 3 summed. Byte 0 is humidity, byte 2 temperature.

PA9 (D8), open-drain with pull-up. Read every 2 s; each read blocks for about 25 ms. Every edge has a 100 µs timeout; a failed read shows DHT11 ERROR.

RTC · CLOCK

On-chip RTC: clock source and set-up

32.768 kHz
LSE crystal

→

÷ 128
PREDIV_A = 127

→

256 Hz
sub-second clock

→

÷ 256
PREDIV_S = 255

→

1 Hz
calendar tick

Fallback: if the crystal does not start within 2.5 s, the internal LSI (about 32 kHz) is used with ÷128 and ÷250. It drifts, so the OLED marks the time LSI.

01
Enable PWR clock; set DBP to unlock the backup domain

02
Write 0xCA then 0x53 to RTC_WPR

03
Set INIT; wait for INITF

04
Write PRER, then TR and DR in BCD, 24-hour

05
Clear INIT, relock, wait for RSF before reading

RTC · PERSISTENCE

Keeping and setting the time

AT EVERY START-UP

Is the RTC enabled, and does BKP0R hold this build’s fingerprint?

Fingerprint = FNV-1a hash of __DATE__ and __TIME__

Yes: plain reset

Leave the running clock alone. The time survives the reset.

No: new build or power loss

Reset the backup domain, start the LSE, set the time from the build timestamp, store the fingerprint.

Limitation: VBAT is tied to VDD on this board, so unplugging USB clears the clock. A coin cell on VBAT would remove this.

Serial commands

```
T 08:30
T 08:30:15
D 27/10/26
?
```

Set time, set time with seconds, set date, show both. Invalid input is rejected with a usage message.

Characters arrive through the USART2 receive interrupt; the main loop assembles and parses each line.

The final build adds R (roster), L (event log as CSV) and S (class status and attendance).

05

Interrupts, interface and results

How button and serial events are handled, the one-button interface with its five screens, the roster and class sessions, how the system was verified, and what we learned.

INTERRUPTS

Three interrupt handlers

`EXTI15_10_IRQHandler`

Blue button B1, PC13

- Falling edge on EXTI line 13.
- Clears the pending bit, ignores edges within 250 ms.
- Only counts presses.
- The loop steps the fan level by the new count.

`EXTI9_5_IRQHandler`

Menu button, D11 (PA7)

- Both edges on EXTI line 7.
- An edge counts only after 30 ms of quiet, so bounce is ignored.
- Low = pressed: store the time. High = released: a tap if not yet held 2 s.
- The loop fires select at 2 s.

`USART2_IRQHandler`

Serial receive

- One interrupt per received byte.
- Reading DR clears RXNE and overrun.
- 64-byte ring buffer; dropped if full.
- Head written only by the handler, tail only by the loop.

Handlers only record events. Every decision is taken in the main loop.

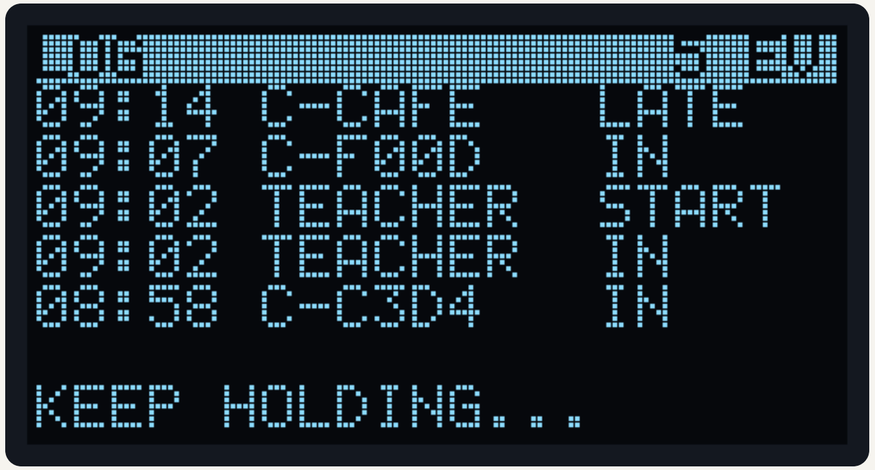
USER INTERFACE

One button, five screens

Control	What it does
D11, tap	Next screen on release: HOME, ROSTER, LOG, CLASS, GRAPH
D11, hold 2 s	The screen’s action, fired while still held
Blue button B1	Fan level 0 → 33 → 66 → 100 → 0, on any screen
Light knob	HOME: brightness. Other screens: pick, scroll or set

Brightness pickup

Back on HOME, the knob is wherever a menu left it. The lights keep their level and HOME shows L75* until the knob reaches that level, so nothing jumps.



After 0.4 s of holding, the bottom line shows KEEP HOLDING... so a hold is never mistaken for a tap.

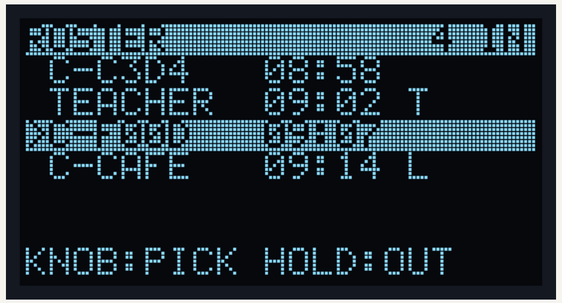
USER INTERFACE

The five screens



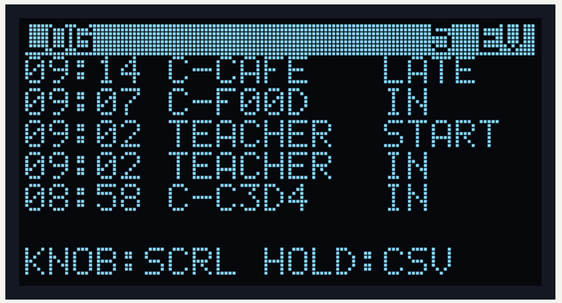
HOME

Live status. Knob: brightness.



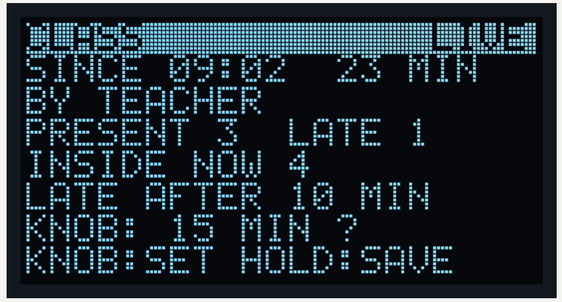
ROSTER

Knob: pick a person. Hold: mark OUT.



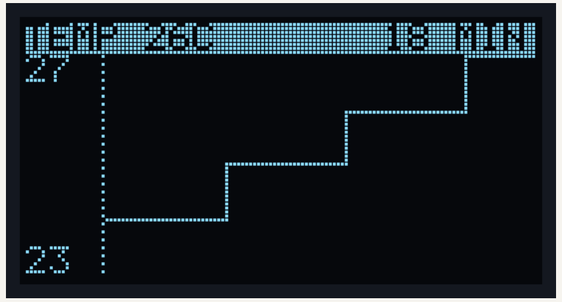
LOG

Knob scrolls. Hold: CSV to serial.



CLASS

Knob: late limit. Hold: save.



GRAPH

Knob: TEMP/HUM/PEOPLE. Hold: freeze.

Same layout everywhere

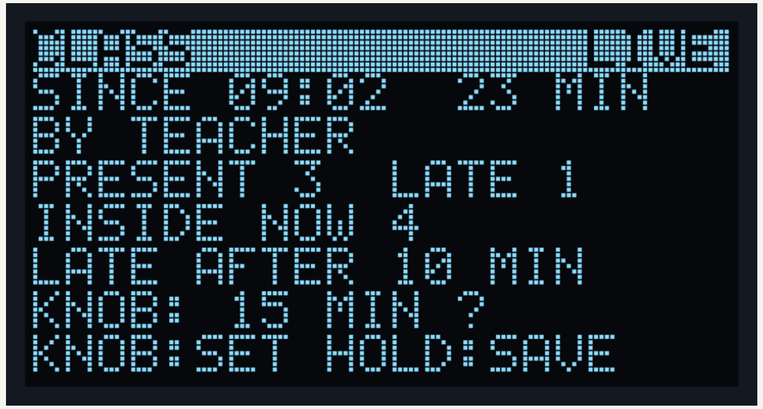
Row 0: inverted title bar. Row 7: what the knob and the hold do here; results of a hold appear there for 2.5 s.

USER INTERFACE · CLASS SESSIONS

Teacher sessions and LATE marking

08:58	C-C3D4 IN	Already inside when the class starts: on time
09:02	TEACHER IN	Card 62E26506 starts the class
09:07	C-F00D IN	5 minutes in: on time
09:14	C-CAFE LATE	12 minutes in, limit 10: marked LATE
09:55	TEACHER OUT	Class ends; serial prints the summary

A student’s first entry decides LATE; leaving and coming back keeps it. Sessions record attendance only; lights and fan behave the same.





USER INTERFACE · DATA

How the roster, log and class are stored

roster_t — logic.h

```
8 slots × {
uid, last_us,
used, inside }
in_sod[8] tap-in time
```

A tap toggles IN and OUT; the same card is ignored for 3 s.

evlog_t — extras.h

```
32 × {
sod, uid, type }
head, count
log_get(0) = newest
```

Ring buffer: when full, the oldest event is overwritten.

session_t — extras.h

```
active, teacher_uid,
start_sod, late_min
att[16] × {
uid, first_sod, late }
```

Plus the last class’s start, end, present and late counts.

CARDS[] in main.c maps a UID to a name and a role. Unnamed cards show as C- plus the last 4 hex digits.

Times are seconds since midnight from the RTC. The late limit is kept in backup register BKP1R through a reset.

USER INTERFACE · DISPLAY

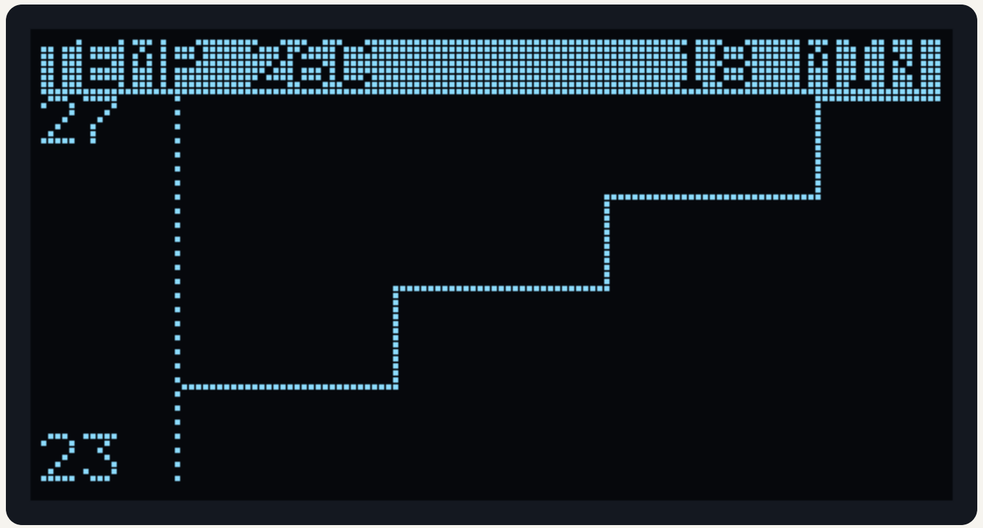
Drawing the screens

01 Draw in memory
fb[8][128]: a 1 KB copy of the 128 × 64 screen. Each pass clears it and draws the current screen: 5 × 7 text, an inverted title bar, graph pixels.

02 Send only what changed
fb_flush() compares each 128-byte page with the copy last sent and sends only the pages that differ over I2C.

03 Keep listening
The RFID reader is polled after every page sent, so drawing never hides a card tap.

SH1106 on I2C1 at 100 kHz, 0x3C or 0x3D. 2 ms timeout per transfer; re-probed every 2 s if it stops answering.



Graphs
108 samples, one every 10 s: the last 18 minutes. Auto-scaled to the data, at least 4 units tall. A failed DHT11 read leaves a gap.

VERIFICATION

Testing on the PC and on the board

551

automated checks on logic.h and extras.h, compiled with gcc and run on a PC

Roster and lockout, light duty, fan levels, kick-start, duty cap, date and time parsing, card names, class sessions and LATE marking, event log, knob steps, brightness pickup and graph scaling. The menu button’s tap and hold logic was also run on the PC with simulated bounce.

Hardware checks

Action	Expected
Stand in zone A for 5 s, LDR covered	Zone A OCC, light on
Leave zone A for 10 s	Zone A clears, light off
Turn the light knob	Brightness follows the knob
Tap a card, then tap again	IN with count 1, then OUT, 0
Tap in, press B1 once	FAN:33 RUN, fan kicks then runs
Tap D11; hold D11 for 2 s	Next screen; screen action
Teacher card, student after the limit	CLASS START; student LATE
Type T 10:15 on serial	OLED shows 10:15:00
Disconnect the DHT11	DHT11 ERROR, loop continues

RESULTS

Memory and timing

17 KB

Flash

Of 512 KB. The Arduino build used about 42 KB.

6.2 KB

RAM

Of 96 KB. 2 KB of it is the screen buffer and its sent copy. No heap.

0.25 s

Loop period

About four passes per second.

3

Interrupts

Two buttons and serial receive; all else polled.

Where one pass goes (estimated)

Zone pings and gaps, ≈125 ms
2 × 60 ms gap + echo time

OLED, up to ≈95 ms
only pages that changed

Rest
ADC, serial

RESULTS

Challenges and how they were resolved

Problem	Cause	Resolution
OLED stayed blank on the Blackpill	No debugger to inspect the board	Moved to the Nucleo with its on-board ST-LINK
OpenOCD could not connect	ST-LINK firmware from 2019	Flashed with STM32_Programmer_CLI over SWD
One zone LED would not light	Breadboard wiring; the fault moved when pins were swapped	Rewired the LED path
Third ultrasonic sensor unreliable	Persistent wiring fault	Design finalised with two zones
OLED froze on an old frame after the pot was added	Not confirmed	Pot removed and USB power cycled; display recovered
Fan speed potentiometer failed	Faulty part	Four fan levels stepped by the blue button
No free pin near D6 for a second menu button	Crowded header	One button on D11: tap = next screen, hold 2 s = select

Most faults were wiring, not firmware. Checking each wire against the pin map, with the Arduino build as a known-good reference, was the fastest diagnostic.

FUTURE SCOPE

Extensions and improvements

Battery-backed clock

A coin cell on VBAT so the time survives loss of USB power.

7-byte UID cards

Add cascade level 2 so newer cards with longer UIDs can be read.

Temperature-led fan

Set the fan speed from the DHT11 reading instead of the knob alone.

Faster flyback diode

A Schottky diode in place of the 1N4007 for 20 kHz switching.

Mains loads

Isolated relay or triac stages to switch real classroom lights and fans.

Networked logging

Send the entry log and room state to a server over a Wi-Fi module.

DEMONSTRATION

Live demonstration sequence

#	Action	What to watch	Who
1	Power the board, type ? on serial	HOME screen; date, time and clock source	Member C
2	Cover the LDR, stand in zone A for 5 s	Zone A shows OCC, light turns on	Member D
3	Press the blue button once, tap a card	FAN:33, IN count 1; fan kicks, then runs	Member A, B
4	Tap the teacher card	CLASS START on serial; CLASS 0M on HOME	Member B
5	Tap D11 through ROSTER, LOG, CLASS	Names, times, T for teacher; events in order	Member D
6	On CLASS: turn knob to 0, hold D11 2 s	SAVED: 0 MIN; the next student is LATE	Member D
7	Tap a second card, open GRAPH	L next to the name; humidity or people curve	Member B, D
8	Tap the teacher card again	CLASS END; summary printed on serial	Member B
9	Tap the other cards out, leave zone A	Fan stops; light off after 10 s	Member B, D

SUMMARY

Two occupancy-sensed zones, RFID attendance with teacher sessions, a PWM fan gated by the headcount, a one-button five-screen interface, temperature, humidity and a real-time clock, all on register-level firmware.

Thank you

Questions are welcome.